Noisy Wordle

Adaptive Algorithms and Analysis in a Corrupted Channel

Team Confident Guessers
Siddharth, L Sai Harsh, Adithya

April 16, 2026

Bachelor of Technology, Indian Institute of Science
E0 259: Data Analytics

Presentation Outline

- **Motivation:** From Real-World Analytics to a Game
- **Problem Statement:** Wordle as a POMDP
- **The Algorithms:** A High-Level Overview
- **Experimental Design:** Simulation Parameters
- **Result Analysis 1:** Benchmarks & Convergence
- **Result Analysis 2:** The Phase Transition of Noise
- **Result Analysis 3:** Learning Custom Language Patterns
- **Conclusion**

Motivation

Motivation: Why Add Noise to a Solved Game?

Sequential Decision Making through data, is a well studied problem. However in many settings, noisy inputs are all we can work with. Many settings face this challenge :

- **Medical Diagnostics:** False positives and false negatives in lab tests.
- **Telecommunications:** Corrupted bits in noisy transmission channels.
- **Robotics:** Inaccurate LiDAR measurements.

These could be due to lack of proper measurement or some uncontrollable errors that creep in.

Abstract Model: Grant Sanderson (3Blue1Brown) popularized solving Wordle using Information Theory (Shannon Entropy). However, his model relies on a crucial assumption: **Perfect Information**. We use this as a backbone and create an abstract model to solve this challenge.

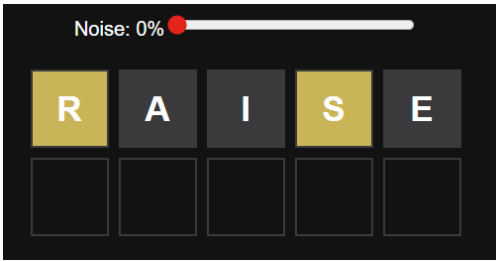
Problem Statement

Problem Statement: The Corrupted Signal

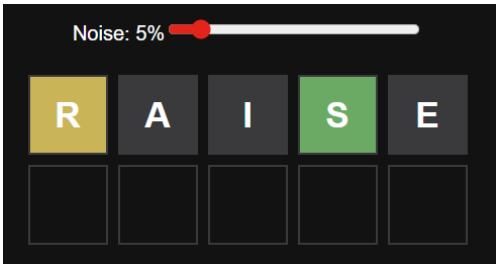
The Shift to a POMDP: Our model has a noise channel acting independently on each feedback tile. The true feedback is corrupted with some noise and each color can be flipped to one of the other 2 with some noise. We model it as a **Partially Observable Markov Decision Process (POMDP)**.

The Core Objective: Identify the hidden target word with 99% confidence using the minimum number of queries, despite the feedback channel corrupting the data.

Problem Statement: The Corrupted Signal



(a) A standard Wordle guess (’raise’) with correct colors.



(b) The exact same guess (’raise’), but with the colors randomly flipped due to noise.

The correct answer is the word ’crops’

The Environment: Modeling the Noise

We define a symmetric, probabilistic noise matrix applied independently to every tile.

Example: The 80/10/10 Split

- **80% Chance:** The tile returns the true, deterministic color.
- **20% Chance (Corruption):** The tile flips to one of the two incorrect colors (10% chance each).

Modular Configuration: This framework helps us to generalize easily based on the underlying problem. The noise levels and environment can easily be tuned to more sophisticated settings.

Theoretical Bounds

Log-likelihood ratio:

$$Z_t(i,j) = \sum_{s=1}^{t} \log \frac{P(F_s \mid g_s, w_i)}{P(F_s \mid g_s, w_j)}$$

$$\mathbb{E}[Z_{t+1} - Z_t] = D_{KL}\big(P(\cdot|g_t, w_i) \,\|\, P(\cdot|g_t, w_j)\big)$$

Information rate:

$$C_i = \max_{g} \min_{j \neq i} D_{KL}\big(P(\cdot|g, w_i) \,\|\, P(\cdot|g, w_j)\big)$$

Lower bound (any strategy):

$$\mathbb{E}_i[T] \;\geq\; \frac{\log(1/\delta)}{C_i}$$

Achievability (Chernoff rule):

$$\mathbb{E}_i[T] \;\lesssim\; \frac{\log(1/\delta)}{C_i}$$

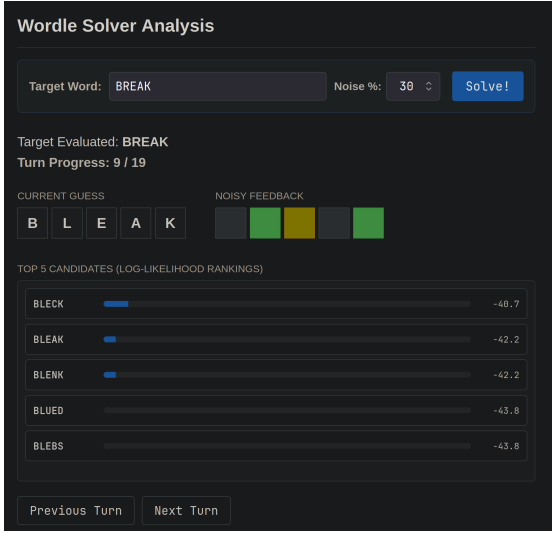
The Algorithms

The Algorithms: A High-Level View

Our solvers navigate the noisy search space using 4 distinct statistical approaches:

- Likelihood Based Greedy Strategy
  - The next guess is the word which has the highest likelihood value.
- Thompson Sampling
  - The next guess is a random word where the probability weights are proportional to likelihood
- Unique Bins - Entropy Based
  - The next guess is the word that separates the contenders into largest number of distinct patterns.

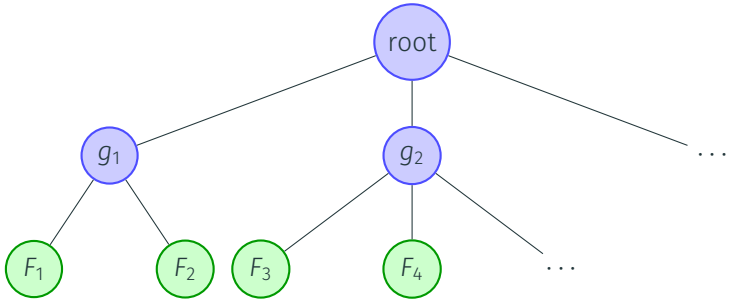
Visualizing the algorithm - Greedy Algorithm



The Algorithms: A High Level Overview

POMCP Deep Search (Simulation Planning)

- Model environment sampled via Monte Carlo rollouts
- Tree search over most likely belief states
- Estimates long-term value of guesses



Experimental Design

Experimental Design: The Setup

To rigorously test our algorithms, we automated thousands of games across four key variables:

- **Scale:** Ran 100 games per algorithm to ensure our results were statistically reliable.
- **Noise Sweeps:** Gradually increased corruption from 0% (perfect information) to 40% noise.
- **Word Lengths:** Pushed beyond the standard game by testing 5, 6, 7, and 8-letter words.
- **Dictionaries:** Compared English words against randomly generated words to see if the models were actually recognizing language patterns.

Datasets & Vocabulary Construction

Data Source & Extraction

- **Source:** Derived from the open-source *dwyl/english-words* GitHub repository (479,000+ public-domain words).
- **English Sets:** Filtered purely alphabetic strings for lengths $n \in \{5, 6, 7, 8\}$ to test if algorithms can exploit latent linguistic structures.
- **Random Sets:** 10,000 uniformly sampled strings (drawn i.i.d. from $\{a\text{–}z\}^n$) to isolate mathematical properties without linguistic bias.

Summary of the datasets

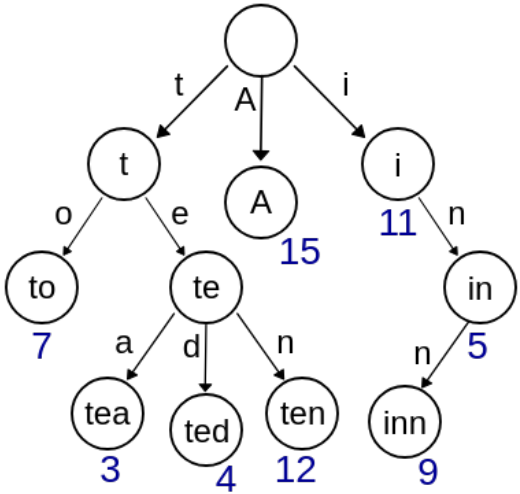
Dataset Summary

Dataset	Type	Length	Vocab Size	Description
english5	Lexical	5	15,918	True English words
english6	Lexical	6	29,874	True English words
english7	Lexical	7	41,997	True English words
english8	Lexical	8	51,626	True English words
random5	Synthetic	5	10,000	Uniformly random strings
random6	Synthetic	6	10,000	Uniformly random strings
random7	Synthetic	7	10,000	Uniformly random strings
random8	Synthetic	8	10,000	Uniformly random strings

Convergence Criteria

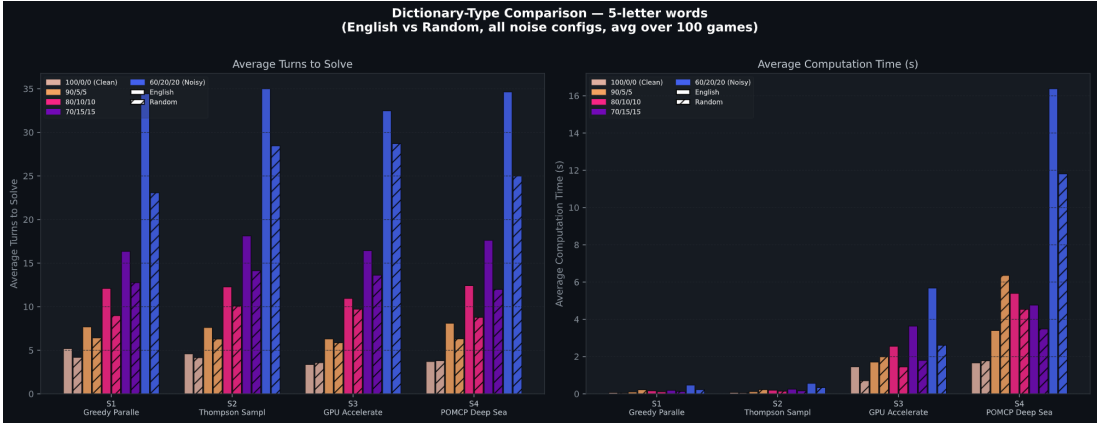
- **Log-Likelihood Scoring:** The solver maintains a running belief score $\ell(w)$ for every candidate word, updated dynamically after receiving noisy feedback.
- **Sequential Probability Ratio Test (SPRT):** Convergence is triggered when the gap (δ) between the #1 and #2 most likely words exceeds a critical threshold (Λ).
- **99% Confidence Target:** We enforced a strict error tolerance of $\alpha = 0.01$. The algorithm formally declares a solution when the evidence gap reaches $\delta \geq \ln\left(\frac{1-\alpha}{\alpha}\right) \approx 4.60$.
- **Hard Cap Failsafe:** To prevent infinite loops caused by severe noise trapping the solver in a local minimum, trials are capped at 100 turns, after which they are recorded as failures.

Optimizations for Efficiency - Trie data structure



Result Analysis

A single test over 5 letter words



Humans Playing the Game

Noise Level	Success Rate	Avg. Guesses
5%	90%	9.2
10%	75%	20.0
15%	25%	34.0
20%	10%	40.0

Our Testers









Result Analysis: The Speed vs. Efficiency Trade-off

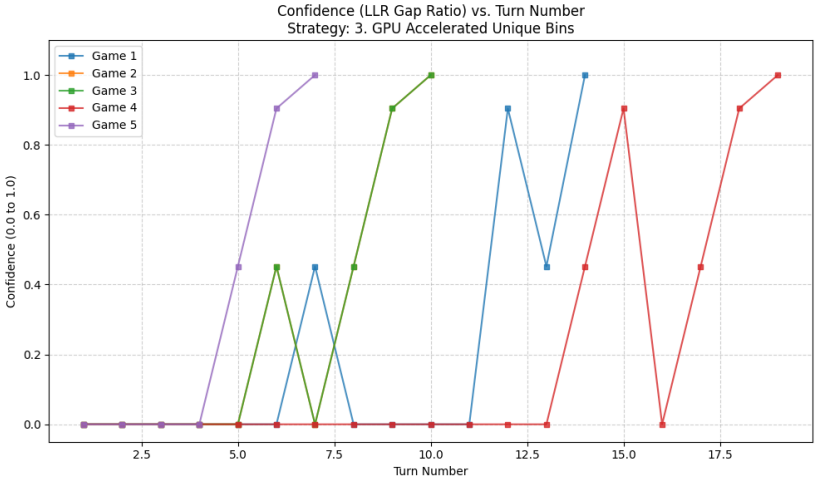
Configuration: 5-letter English words, 80/10/10 noise, 100 game averages.

Strategy	Success Rate	Avg Turns	Avg Time (s)
Greedy Parallel Trie	100.0%	12.09	0.1610
Thompson Sampling	100.0%	12.26	0.1931
GPU Accelerated Bins	99.0%	10.94	2.5509
POMCP Deep Search	99.0%	12.41	5.3896

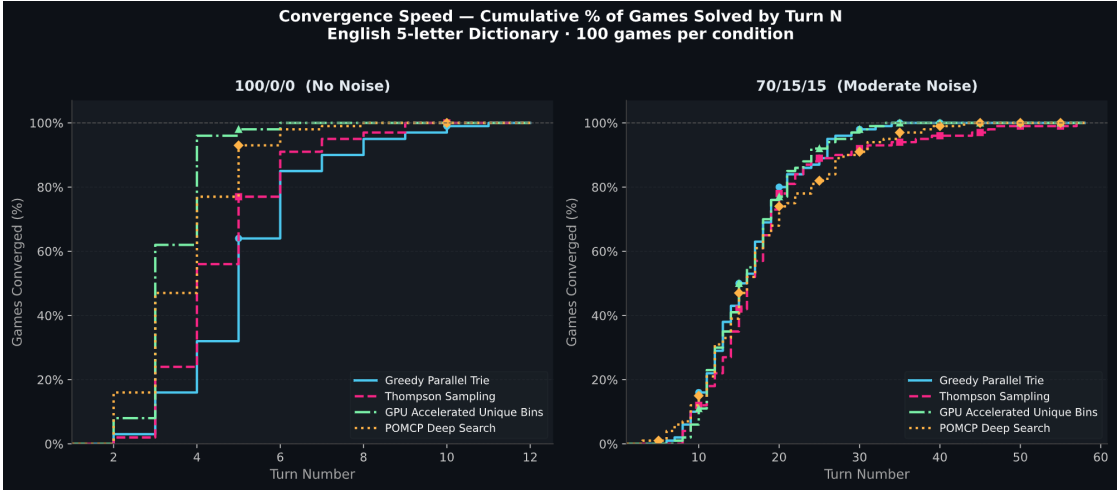
Key Insight: Under moderate noise, the **Greedy Parallel Trie** offers the best practical performance, executing in fractions of a second with a 100% success rate.
While **GPU Accelerated Unique Bins** is slightly more sample-efficient (saving ~ 1 turn), it incurs a $15\times$ computational overhead and a marginal drop in reliability.

Result Analysis: Tracking Algorithm Confidence

Setup: 5-Letter English Words | 80/10/10 Noise | GPU Accelerated Bins



Convergence Analysis: Perfect vs. Noisy Info

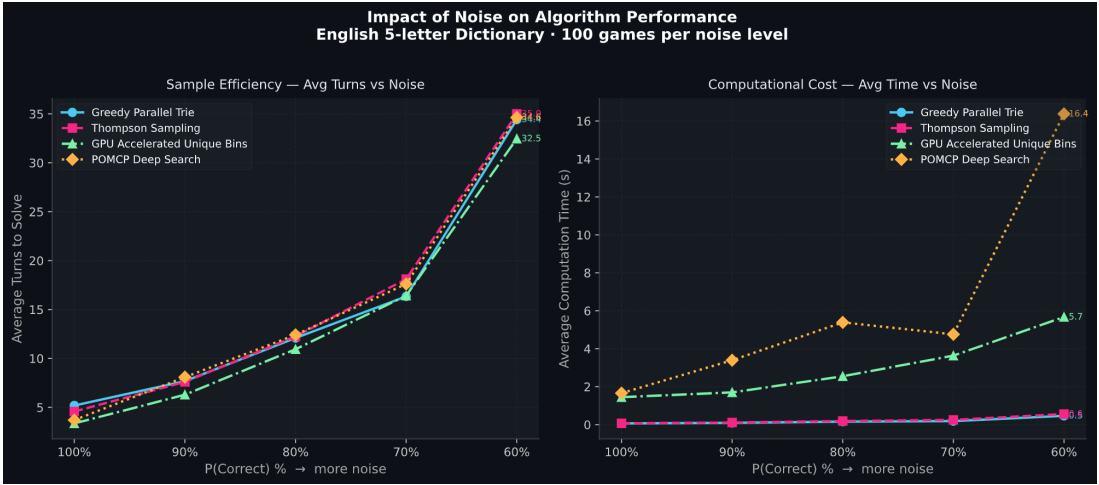


Convergence Analysis: Key Takeaways

Speed to 99% Confidence

- **Perfect Info (100/0/0):** Clear differences. **GPU Unique Bins** is highly efficient, solving ∼100% by Turn 6.
- **Noise Penalty (70/15/15):** A massive scale shift. Full convergence requires nearly 60 turns across all models.
- **Diminished Information Gain:** The jumps (which act as a proxy for information gained) is becoming more compressed with addition of noise.

Impact of Noise on Turn Count and Computational Cost

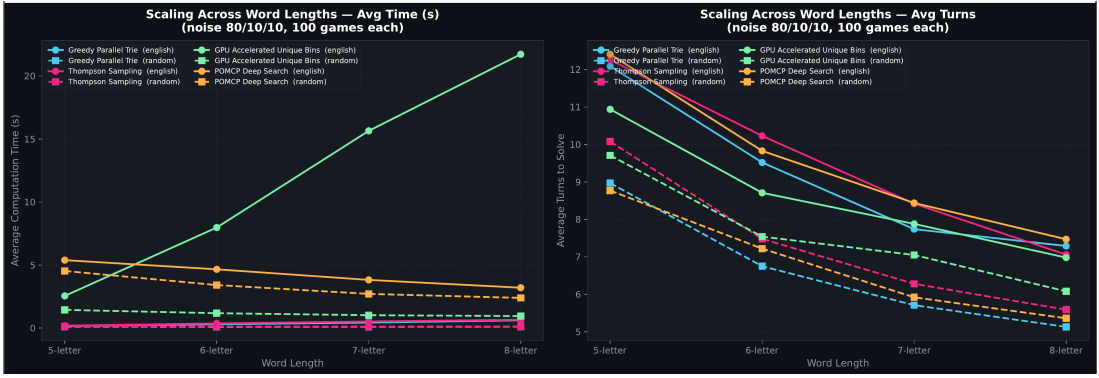


Impact of Noise: Key Takeaways

Turn Count vs. Compute Time

- **Sample Efficiency:** Turn counts rise exponentially with noise (irrespective of algorithm). **GPU Unique Bins** performs slightly better due to entropy minimization.
- **Stable Compute:** **Greedy Trie** and **Thompson Sampling** maintain flat computation times ($\mathcal{O}(|\mathcal{D}|)$) regardless of noise.
- **Compute Explosion:** Costs for **GPU Unique Bins** and **POMCP** increase with noise%. Noisy feedback leaves larger candidate pools, forcing heavier matrix math and much deeper tree searches.

Scaling: Word Lengths & Dictionaries



Scaling: Key Takeaways

Linguistic Patterns vs. Random Strings

- **Length Advantage:** Longer words actually require *fewer* turns. The combinatorial space is sparser, naturally reducing competing hypotheses.
- **The English Penalty:** Algorithms solve Random strings faster. Real English words have dense orthographic clusters (e.g., "-IGHT") that amplify noise confusion.
- **Memory Bottleneck:** For English sets $> 50{,}000$ words (8-letters), **GPU Unique Bins** compute time spikes massively. Other strategies scale with negligible penalty.

Summary of the datasets - revisit

Dataset Summary

Dataset	Type	Length	Vocab Size	Description
english5	Lexical	5	15,918	True English words
english6	Lexical	6	29,874	True English words
english7	Lexical	7	41,997	True English words
english8	Lexical	8	51,626	True English words
random5	Synthetic	5	10,000	Uniformly random strings
random6	Synthetic	6	10,000	Uniformly random strings
random7	Synthetic	7	10,000	Uniformly random strings
random8	Synthetic	8	10,000	Uniformly random strings

Conclusion

Conclusion: Key Findings

- **A New Paradigm:** Introducing heavy noise transforms Wordle from a simple game into a complex probabilistic challenge.
- **Efficiency vs. Bottlenecks: GPU Unique Bins** minimizes turns effectively but hits memory limits on very large datasets.
- **Speed vs. Variance:** The **Greedy Parallel Trie** is the fastest algorithm, but struggles with high variance and turn inflation under severe noise.
- **The Linguistic Penalty:** Dense orthographic clusters in English words make noisy feedback significantly harder to decipher compared to mathematically random strings.

Future Directions

- **Real-World Deployment:** Applying these algorithms to practical, noisy sequential decision-making scenarios beyond abstract games.
- **Advanced Scaling:** Expanding datasets and utilizing advanced sampling techniques to uncover deeper performance patterns.
- **Complex Noise Models:** Transitioning from independent uniform noise to Markovian or correlated, non-independent error environments.

Play the Game:
noisy-wordle.web.app

Open Source Repository:
github.com/adithyaka3/Noisy-Wordle

Scan to Play



Thank You!

Questions?